

UNIVERSIDAD SAN CARLOS DE GUATEMALA
CENTRO UNIVERSITARIO DE OCCIDENTE
DIVISIÓN DE CIENCIAS DE LA INGENIERÍA
CARRERA DE CIENCIAS Y SISTEMAS
SEGUNDO SEMESTRE 2026
ORGANIZACIÓN DE LENGUAJES Y
COMPILADORES 2
ING. MARIO RAMIREZ



MANUAL TÉCNICO PROYECTO FASE
(Nexus Solairy)

Erikson Denilson Orozco Monterroso 202131770

Quetzaltenango, 25 de Septiembre de 2026

CONTENIDO

CONTENIDO.....	2
1. Introducción.....	4
1.1 Alcance del documento.....	4
2. Objetivos.....	4
2.1 Objetivo general.....	4
1. 2.2 Objetivos específicos.....	5
3. Tecnologías utilizadas.....	5
4. Arquitectura y generación de Código de Tres Direcciones (C3D).....	6
Figura 1. Pipeline de análisis y traducción de Nexus Solairy.....	7
4.1 Organización en paquetes.....	7
5. Especificación léxica.....	8
5.1 Canales ocultos (comentarios y espacios).....	9
5.2 Palabras reservadas por lenguaje.....	9
5.3 Operadores y símbolos.....	10
5.4 Literales.....	11
6. Especificación sintáctica.....	12
6.1 Estructura general de programas por lenguaje.....	12
6.2 Declaraciones por lenguaje.....	12
6.3 Funciones por lenguaje.....	13
6.4 Sentencias y estructuras de control por lenguaje.....	14
6.5 Expresiones y precedencia por lenguaje.....	15
6.6 Estructuras y arreglos por lenguaje.....	16
6.7 Entrada / salida por lenguaje.....	16
7. Especificación semántica.....	17
7.1 Jerarquía de tipos.....	17
7.2 Reglas de compatibilidad y promoción de tipos.....	17
7.3 Tabla de compatibilidad — resultado de operación binaria.....	18
7.4 Tabla de símbolos y ámbitos (scope).....	19
7.5 Validaciones semánticas obligatorias.....	19
7.6 Clasificación de errores.....	20
8. Diagrama de clases.....	21
Figura 2. Diagrama de clases de Nexus Antiquus.....	21
9. Estructura del proyecto.....	21
10. Requisitos del sistema y manual de compilación.....	23
10.1 Requisitos.....	23
10.2 Ejecución en desarrollo (Maven).....	23
11.3 Construcción del fat-JAR.....	23
10.4 Empaquetado nativo con jpackage.....	24
10.5 Ejecución de la aplicación empaquetada.....	24

11. Glosario de términos.....	25
Anexo A. Gramática completa (ANTLR4).....	27
A.1 Gramáticas de Zetariano (ZetarianoLexer.g4 y ZetarianoParser.g4).....	27
A.2 Gramáticas de Y? (YLexer.g4 y YParser.g4).....	30
A.3 Gramáticas de PigLatin (PigLatinLexer.g4 y PigLatinParser.g4).....	41
Anexo B. Diagrama Extra.....	42
B.1 Diagrama de estados — Pila del parser (SRA).....	42

1. Introducción

Nexus Solairy es un Entorno de Desarrollo Integrado (IDE), compilador de Código de Tres Direcciones (C3D), máquina virtual e intérprete desarrollado en Java para el lenguaje Codex Latinus (.lat). En esta Fase, el sistema evoluciona para soportar la compilación completa a Código de Tres Direcciones (C3D), optimización mediante entornos de memoria y Stack/Heap virtual, ejecución en una Máquina Virtual dedicada y traducción multilenguaje hacia tres lenguajes objetivo: Y?, Zetariano y PigLatin.

1.1 Alcance del documento

- La documentación cubre los siguientes aspectos:
 - Arquitectura general del pipeline de análisis (léxico, sintáctico, semántico, AST, traducción y generación de C3D).
 - Especificación formal de la gramática (tokens, palabras reservadas, símbolos y reglas de producción).
 - Reglas semánticas: jerarquía de tipos, compatibilidad, ámbitos (scopes) y tabla de símbolos.
 - Generación de Código de Tres Direcciones (C3D), representación mediante cuartetos (Quadruple) y ejecución en la Máquina Virtual Solairy (C3DVirtualMachine).
 - Traducción multilenguaje hacia Y?, Zetariano y PigLatin, además de transpilación a C99 y compilación ejecutable mediante GCC.
 - Manual de compilación, ejecución y empaquetado.
 - Batería de pruebas y criterios de validación.

2. Objetivos

2.1 Objetivo general

Validar un lenguaje de programación como parte del proceso de compilación, aplicando los conocimientos adquiridos en la clase magistral sobre análisis léxico, sintáctico y semántico.

1. 2.2 Objetivos específicos

2. Implementar la gramática del lenguaje Codex Latinus.
3. Adaptarse a la herramienta ANTLR4 para la generación de lexers y parsers.
4. Generar un AST coherente y una Tabla de Símbolos que registre correctamente los tipos y el alcance (scope) de variables y funciones.
5. Verificar la consistencia semántica del código, reportando cualquier anomalía antes de cualquier proceso de transformación.

3. Tecnologías utilizadas

Componente	Tecnología	Propósito
Lenguaje base	Java 21 (JDK)	Implementación de todo el compilador, IDE e intérprete
Generador léxico/sintáctico	ANTLR 4.13.2	Generación del Lexer y Parser a partir de la gramática (.g4)
Interfaz gráfica	JavaFX	IDE de escritorio: editor, visualizadores y consola
Editor de código	RichTextFX (CodeArea)	Resaltado de sintaxis y numeración de líneas
Componentes UI	ControlsFX / Ikonli / BootstrapFX	Controles adicionales e iconografía
Gestor de dependencias	Apache Maven 3.8+	Compilación, gestión de dependencias y empaquetado

Componente	Tecnología	Propósito
Empaquetado	maven-shade-plugin, jpackage	Generación de fat-JAR e imagen nativa autocontenida
Visualización de grafos	Graphviz	Apoyo en el cálculo/representación del AST
Compilador C Nativo	GCC (GNU Compiler Collection)	Compilación del código C99 generado desde C3D a ejecutables binarios
Servicio en Internet que almacena repositorios Git.	GitHub	Servicio que alojó el proyecto.
Sistema de control de versiones.	Git	Sistema de control de versiones durante el desarrollo del proyecto.
Sistema Operativo	CachyOS	Sistema operativo usado para el desarrollo del proyecto.
Diagramas a base de script.	PlantUML	Herramienta externa open source que ayude a realizar los diagramas presentados en el manual.

4. Arquitectura y generación de Código de Tres Direcciones (C3D)

El pipeline de Nexus Solairy procesa el código fuente realizando análisis léxico, sintáctico y semántico, generando un Árbol de Sintaxis Abstracta (AST). A partir del AST, el sistema genera Código de Tres Direcciones (C3D) intermedio compuesto por instrucciones cuádruplas (Quadruple: op, arg1, arg2, result), manejo del Stack (pila de llamadas/entornos) y Heap (memoria dinámica para arreglos y estructuras), administración de punteros (P y H) y etiquetas temporales. Este C3D puede ser ejecutado directamente en la Máquina Virtual

Solairy (C3DVirtualMachine), traducido a C99 y compilado ejecutable con GCC, o traducido a los lenguajes destino (Y?, Zetariano y PigLatin).

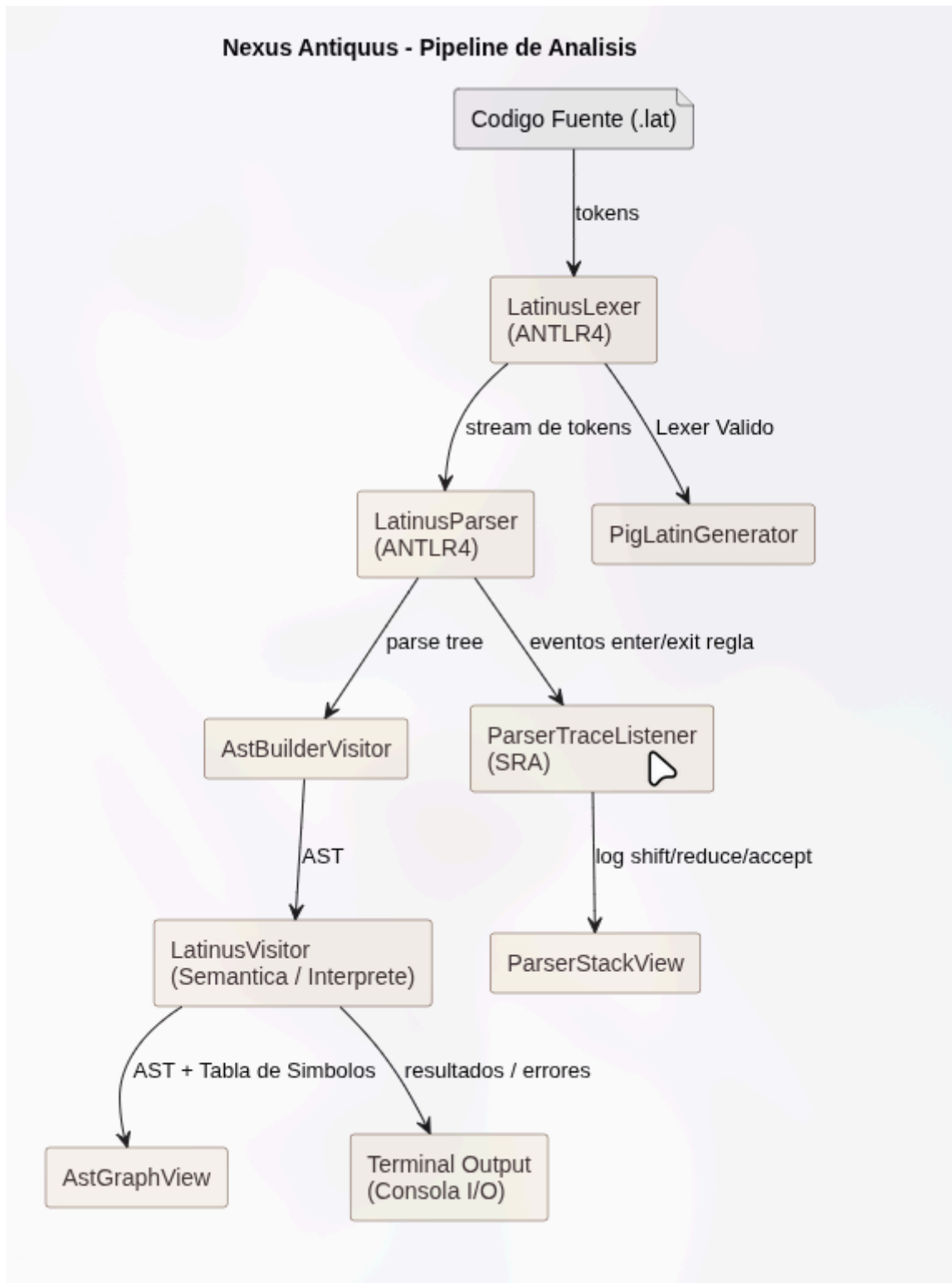


Figura 1. Pipeline de análisis y traducción de Nexus Solairy

4.1 Organización en paquetes

El código fuente Java se organiza en los siguientes paquetes bajo `com.example.nexussolairy`:

Paquete	Responsabilidad
analysis	Orquestación del pipeline completo (AnalysisPipeline)
lexer	Servicios y listeners léxicos sobre LatinusLexer
parser	Servicios de parseo y trazado de la pila (ParserTraceListener)
ast	Definición del AST (ASTNode, ASTNodeKind) y su construcción (AstBuilderVisitor)
visitor	Intérprete: evaluación de sentencias y expresiones, verificación semántica para los tres lenguajes
model	Modelos de datos: errores, tokens, símbolos, tipos y estructuración de la representación intermedia
c3d	Generador de C3D (C3DGenerator), representación por cuartetos (Quadruple) y Máquina Virtual (C3DVirtualMachine)
ui	Componentes visuales: AstGraphView, ParserStackView, C3DView
utils	Utilidades: verificador de tipos, proveedores de I/O y transpilación C99/GCC
controller	Controlador MVC de la interfaz (MainController)

5. Especificación léxica

Las especificaciones léxicas de los lenguajes están definidas en sus respectivos archivos de lexer: ZetarianoLexer, YLexer y PigLatinLexer. Todos los lenguajes son sensibles a

mayúsculas y minúsculas (case-sensitive) y procesan sus tokens de acuerdo con sus propias estructuras sintácticas.

5.1 Canales ocultos (comentarios y espacios)

Lenguaje	Token	Patrón	Descripción
Zetariano	LINE_COMMENT / BLOCK_COMMENT	// ... / ## ... ##	Comentarios de línea y bloque (canal HIDDEN)
Y?	LINE_COMMENT / BLOCK_COMMENT	// ... / ## ... ##	Comentarios de línea y bloque (canal HIDDEN)
PigLatin	LINE_COMMENT / BLOCK_COMMENT	// ... / ## ... ##	Comentarios de línea y bloque (canal HIDDEN)
Todos	WS	[\t\r\n]+	Espacios en blanco, tabulaciones y saltos de línea (canal HIDDEN)

5.2 Palabras reservadas por lenguaje

Lenguaje	Categoría	Token	Lexema / Palabras clave
Zetariano	Estructura y Control	ZETA_VAR, ZETA_MAIN, SI, ALITER, etc.	ZETA_VARIABILES, ZETA_MAIOR, si, aliter, dum, facere, per, interrompe, perge
Zetariano	Tipos y Funciones	NUMERUS, TEXTUM, ACTIO, RATIO, REDDERE	numerus, textum, decimalis, littera, bool, actio, ratio, reddere
Y?	Estructura Interrogativa	Y_VAR, Y_MAIN, Y_IF, Y_WHILE, etc.	Palabras reservadas adaptadas al paradigma interrogativo Y?
PigLatin	Estructura Porcina	PIG_VAR, PIG_MAIN, OINK, OINK_OINK, etc.	Comandos con sufijos pig-latin y funciones especiales %OINK, %OINK_OINK

5.3 Operadores y símbolos

Token	Símbolo	Categoría
EQ / NEQ	== / !=	Relacional (igualdad)
LE / GE / LT / GT	<= / >= / < / >	Relacional (orden)
AND / OR	&& /	Lógico
INC / DEC	++ / --	Incremento / decremento abreviado
READ	<<	Entrada de consola

Token	Símbolo	Categoría
PRINT	>>	Salida de consola
PLUS / MINUS / MULT / DIV	+ - * /	Aritmético
ASSIGN	=	Asignación
COLON / SEMI / COMMA / DOT	: ; , .	Puntuación
LBRACE / RBRACE	{ }	Delimitadores de bloque
LBRACK / RBRACK	[]	Delimitadores de arreglo/índice
LPAREN / RPAREN	()	Delimitadores de expresión/parámetros

5.4 Literales

Token	Patrón (regex)	Ejemplo
DECIMAL	[0-9]+ '.' [0-9]+	9.81
NUMBER	[0-9]+	10
STRING	" (ESC ~["\\r\n])* "	"Somos la resistencia"
CHAR	' (ESC ~["\\r\n]) '	'a'
IDENTIFIER	[a-zA-Z_][a-zA-Z0-9_]*	mi_entero, fuerza

El fragmento ESC define las secuencias de escape admitidas dentro de cadenas y caracteres:

```
\b \t \n \r \f \" \' \\.
```

6. Especificación sintáctica

Las reglas sintácticas para cada uno de los lenguajes están definidas en sus respectivos parsers de ANTLR4: ZetarianoParser.g4, YParser.g4 y PigLatinParser.g4. La regla principal de entrada para cada compilador/traductor consume el flujo de tokens generado por el lexer correspondiente hasta alcanzar EOF.

6.1 Estructura general de programas por lenguaje

Cada lenguaje posee una estructura declarativa y de ejecución adaptada a su gramática:

```
// Zetariano

zetarianoProgram : variableSection? functionSection? mainSection FINIS_PROG SEMI ;


// Y?

yProgram : yVariableSection? yFunctionSection? yMainSection Y_FINIS SEMI ;


// PigLatin

pigLatinProgram : pigVariableSection? pigFunctionSection? pigMainSection PIG_FINIS
SEMI ;
```

6.2 Declaraciones por lenguaje

Las reglas de declaración en ZetarianoParser, YParser y PigLatinParser permiten la definición de variables globales, arreglos y estructuras personalizadas según la sintaxis particular de cada uno.

declaration : variableDeclaration | arrayDeclaration | structDeclaration

;

variableDeclaration

: ESTO IDENTIFIER COLON type expression? SEMI

| ESTO IDENTIFIER COLON booleanLiteral SEMI

| ESTO IDENTIFIER COLON IDENTIFIER structInit

| ESTO IDENTIFIER COLON IDENTIFIER expression SEMI

;

arrayDeclaration

: SERIES IDENTIFIER LBRAK expression RBRAK COLON type arrayInit? SEMI

| SERIES IDENTIFIER LBRAK expression RBRAK COLON arrayInit SEMI

;

structDeclaration

: STRUCTURA IDENTIFIER LBRAK structField* RBRAK FINIS SEMI

;

6.3 Funciones por lenguaje

functionDeclaration : voidFunction | returnFunction

;

voidFunction

: ACTIO IDENTIFIER LPAREN parameterList? RPAREN functionBody FINIS SEMI

;

returnFunction

: RATIO type IDENTIFIER LPAREN parameterList? RPAREN functionBody FINIS SEMI

;

functionBody : LBRACE functionVarSection? statement* RBRACE

;

functionVarSection : VARIABLES LBRACK declaration* RBRACK

;

En los tres lenguajes (Zetariano, Y? y PigLatin), las funciones soportan procedimientos sin retorno y funciones con retorno explícito, manteniendo aislamiento de variables locales en sus secciones internas correspondientes.

6.4 Sentencias y estructuras de control por lenguaje

statement

: localDeclaration | assignmentStatement | ioStatement

| ifStatement | whileStatement | doWhileStatement | forStatement

| jumpStatement | returnStatement | block | expressionStatement

;

ifStatement

: SI LPAREN expression RPAREN block

(ALITER LPAREN expression RPAREN block)*

(ALITER block)?

FINIS SEMI

;

whileStatement : DUM LPAREN expression RPAREN block FINIS SEMI

;

doWhileStatement : FACERE block DUM LPAREN expression RPAREN SEMI

;

forStatement : PER LPAREN forInit SEMI expression SEMI expression RPAREN block

;

jumpStatement : INTERRUPE SEMI | PERGE SEMI

;

returnStatement : REDDERE expression? SEMI

;

6.5 Expresiones y precedencia por lenguaje

La precedencia de operadores se mantiene uniforme en todos los parsers (ZetarianoParser, YParser y PigLatinParser) mediante una estratificación equivalente de reglas de expresión.

Nivel (menor→mayor)	Regla	Operadores
1	orExpression	
2	andExpression	&&
3	relationalExpression	== != < > <= >=
4	additiveExpression	+ -
5	multiplicativeExpression	* /
6	unaryExpression	non, - (unario)
7	postfixExpression	++ -- (postfijo)
8	primaryExpression	literales, identificadores, llamadas, ()

6.6 Estructuras y arreglos por lenguaje

Las definiciones de estructuras y arreglos se adaptan gramaticalmente en cada dialecto, garantizando el acceso por índice y por atributo.

6.7 Entrada / salida por lenguaje

```
// Zetariano: PRINT expression (PRINT expression)* SEMI ; / READ ;
// Y?: Y_PRINT yExpression SEMI ; / Y_READ ;
// PigLatin: OINK pigExpression SEMI ; / OINK_OINK ;
```

7. Especificación semántica

El análisis semántico se ejecuta después de que el árbol de sintaxis haya sido construido, recorriendo el AST mediante *LatinusVisitor*. Sus responsabilidades son: construir la tabla de símbolos con ámbitos anidados, verificar la compatibilidad de tipos (type checking) y detectar anomalías de control de flujo antes de permitir cualquier traducción o ejecución, si existen errores léxico o sintácticos no se ejecuta el código.

7.1 Jerarquía de tipos

Los tipos primitivos se ordenan en una jerarquía numérica utilizada para resolver el tipo resultante de cualquier operación binaria: se toma siempre la jerarquía más alta entre los dos operandos.

Tipo	Palabra clave	Jerarquía	Descripción
Texto	textum	5	Cadena de caracteres delimitada por comillas dobles
Decimal	decimalis	4	Punto flotante de 64 bits
Entero	numerus	3	Entero de 64 bits
Carácter	littera	2	Carácter individual delimitado por comillas simples
Booleano	bool	1	verum (verdadero) / falsus (falso)
Estructura	structura	0	Tipo compuesto definido por el usuario (no operable)
Vacío	void	0	Tipo de retorno de procedimientos actio

7.2 Reglas de compatibilidad y promoción de tipos

- Al operar dos tipos primitivos distintos, el resultado adopta el tipo de mayor jerarquía ejemplo:

numerus + decimalis = decimalis

littera + numerus = numerus (littera como código ascii)

- El tipo textum es incompatible con operaciones aritméticas y relacionales excepto la suma (+), que se interpreta como concatenación ejemplo:

textum + littera = textum

- Los arreglos (series) y las estructuras (structura) no son tipos primitivos y no pueden operarse directamente entre sí ni con otros tipos; sin embargo, los valores que contienen (al acceder por índice o por atributo) sí son primitivos y pueden participar en expresiones.
- Toda condición evaluada en si, dum, facere...dum y la segunda cláusula de per debe ser estrictamente de tipo booleano. Evaluar un valor numérico (u otro tipo no booleano) como condición se reporta como error semántico.
- Los índices de arreglos pueden ser expresiones; si el índice es evaluable en tiempo de análisis semántico y se encuentra fuera del rango declarado, se reporta un error.

7.3 Tabla de compatibilidad — resultado de operación binaria

Operando A	Operando B	Resultado	Observación
numerus	numerus	numerus	Aritmética estándar
numerus	decimalis	decimalis	Promoción a la jerarquía más alta
numerus	littera	numerus	littera se promueve a numerus (código)
decimalis	littera	decimalis	Promoción a decimalis

Operando A	Operando B	Resultado	Observación
textum	numerus / decimalis / littera / bool	textum	Solo válido con '+' → concatenación
textum	textum	textum	Concatenación
bool	bool	bool	Solo válido con operadores lógicos (&&,) o relacionales
structura	cualquiera	ERROR	Tipo no operable directamente
series	cualquiera	ERROR	Tipo no operable directamente (usar acceso por índice)

7.4 Tabla de símbolos y ámbitos (scope)

La tabla de símbolos se organiza en ámbitos anidados que reflejan la estructura léxica del programa:

- **Ámbito global:** variables declaradas en la sección VARIABILES> del programa.
- **Ámbito de función y local:** parámetros y variables locales declaradas en VARIABILES[...] dentro de cada función; visible únicamente dentro de esa función.
- **Ámbito local:** cada bloque { ... } (cuerpos de si, dum, facere, per) puede introducir su propio ámbito anidado sobre el ámbito que lo contiene, permitiendo resolución de identificadores por cercanía (de adentro hacia afuera).

Cada entrada de la tabla de símbolos registra, como mínimo: nombre del identificador, tipo, categoría (variable, arreglo, estructura, parámetro, función), ámbito de declaración y, para funciones, la lista de tipos de parámetros y el tipo de retorno.

7.5 Validaciones semánticas obligatorias

1. Todo identificador debe estar declarado antes de su uso; el uso de identificadores no declarados se reporta como error.
2. No se permiten redeclaraciones del mismo identificador dentro de un mismo ámbito.
3. Los atributos de una estructura no pueden repetir nombre dentro de la misma definición.
4. Al instanciar una estructura, todos los atributos deben recibir un valor explícito y del tipo adecuado; el orden de los atributos en el literal no importa.
5. En funciones con retorno (ratio), el tipo del valor retornado debe ser compatible con el tipo declarado en la firma, no permite cambio de jerarquía.
6. Toda función con retorno debe garantizar que existe un valor de retorno en todos los caminos posibles de ejecución (análisis de alcanzabilidad sobre las ramas si/aliter).
7. No se permite código no rastreable (unreachable) después de una sentencia reddere dentro del mismo bloque.
8. Las sentencias interrumpere y perge solo son válidas dentro del cuerpo de un ciclo (dum, facere...dum, per); su uso fuera de un ciclo es un error semántico.

7.6 Clasificación de errores

Categoría	Momento de detección	Ejemplos
Léxico	LatinusLexer	Carácter no reconocido, cadena/carácter sin cerrar
Sintáctico	LatinusParser	Token inesperado, falta de ';' o de 'finis'
Semántico	LatinusVisitor / Type Checker	Tipos incompatibles, identificador no declarado, retorno faltante, condición no booleana, índice fuera de rango

Todo error reportado incluye, como mínimo, fila, columna, tipo de error y una descripción legible del problema, siguiendo el formato de salida INFO> / OK> / ERROR> descrito en la sección de la consola interactiva.

8. Diagrama de clases

El siguiente diagrama resume las clases principales del sistema y sus relaciones, agrupadas por paquete.

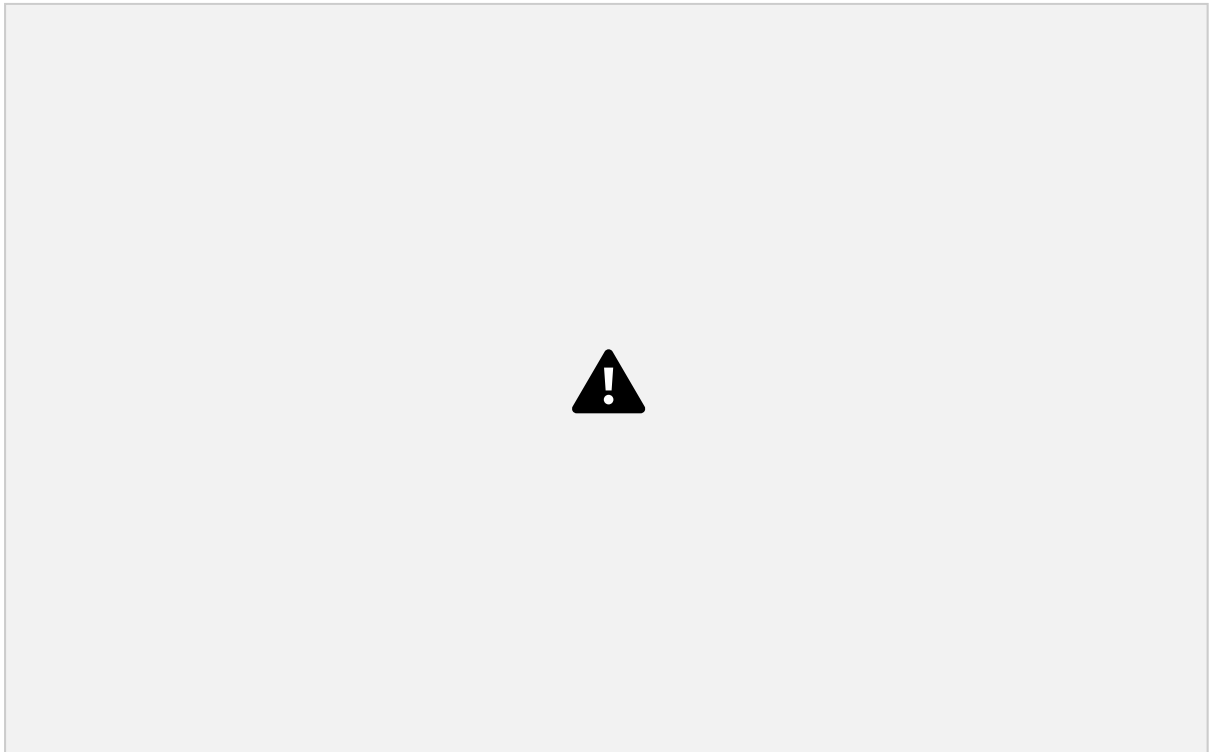


Figura 2. Diagrama de clases de Nexus Antiquus.

9. Estructura del proyecto

```
nexus-solairy/  
├── pom.xml                # Configuración de  
dependencias y plugins Maven  
├── mvnw / mvnw.cmd       # Wrappers de Maven  
└── src/  
    ├── main/  
    │   ├── antlr4/org/nexus/nexussolairy/  
    │   │   ├── YLexer.g4    # Gramática léxica  
    │   │   para lenguaje Y?  
    │   │   ├── YParser.g4   # Gramática  
    │   │   sintáctica para lenguaje Y?  
    │   │   └── ZetarianoLexer.g4 # Gramática léxica  
    │   para lenguaje Zetariano
```

		└─ ZetarianoParser.g4	# Gramática
sintáctica para lenguaje Zetariano			
		└─ PigLatinLexer.g4	# Gramática léxica
para lenguaje PigLatin			
		└─ PigLatinParser.g4	# Gramática
sintáctica para lenguaje PigLatin			
		└─ java/	
		└─ module-info.java	# Configuración
modular de JavaFX y dependencias			
		└─ org/nexus/nexussolairy/	
		└─ Launcher.java	# Entry point para
ejecución Fat-JAR			
		└─ NexuSolairyApp.java	# Entrada principal
de la aplicación JavaFX			
		└─ backend/	
		└─ c/	# Transpilador C
(CSourceCodeGenerator) y servicio GCC			
		└─ vm/	# Máquina Virtual
C3D (C3DVirtualMachine)			
		└─ c3d/	# Generación de
cuartetos (YC3DVisitor, ZetarianoC3DVisitor, MemoryLayout)			
		└─ controller/	# Controlador de la
interfaz gráfica (MainController)			
		└─ model/	# Modelos de C3D,
análisis semántico, tokens y vistas			
		└─ patron/	# Factorías
(LexerFactory, ParserFactory, VisitorFactory)			
		└─ service/	
		└─ analysis/	# Orquestación del
pipeline (AnalysisPipeline, ImportResolver)			
		└─ grammar/	# Servicios léxicos
y listeners de error			
		└─ parser/	# Servicios
sintácticos y YIndentationLexer			
		└─ ui/	# Gestión de
archivos, proyectos y workspace			
		└─ ui/	# Resaltado léxico y
temas (SyntaxColor)			
		└─ utils/	# Clases auxiliares y
contenedores de resultados			
		└─ view/	# Diálogos modales y
componentes visuales			

```

    |      └─ visitor/                                # Intérpretes
semánticos y evaluadores por lenguaje
    └─ resources/org/nexus/nexussolairy/
        └─ view/MainView.fxml                        # Definición de la
interfaz gráfica en FXML
        └─ styles/styles.css                          # Hojas de estilo CSS
del IDE

```

10. Requisitos del sistema y manual de compilación

10.1 Requisitos

Requisito	Versión mínima
Java Development Kit (JDK)	21 o superior (OpenJDK 21, Eclipse Temurin 21)
Apache Maven	3.8+ (o el wrapper incluido ./mvnw)
jpackage	Incluido en JDK 21+
Sistema operativo	Linux (Debian/Ubuntu/Fedora), macOS o Windows

10.2 Ejecución en desarrollo (Maven)

Para compilar las gramáticas ANTLR4 y ejecutar la interfaz en modo de desarrollo:

```

./mvnw clean javafx:run

# o, con Maven instalado en el PATH:

mvn clean javafx:run

```

11.3 Construcción del fat-JAR

El proyecto utiliza maven-shade-plugin para empaquetar todas las dependencias (JavaFX, ANTLR4, RichTextFX, BootstrapFX, etc.) en un único JAR ejecutable, usando Launcher como clase principal:

```
./mvnw clean package

# genera: target/nexus-solairy-1.0-SNAPSHOT.jar

java -jar target/nexus-solairy-1.0-SNAPSHOT.jar
```

10.4 Empaquetado nativo con jpackage

```
# 1. Limpiar distribuciones previas y compilar el JAR

rm -rf dist

./mvnw clean package

# 2. Generar el paquete nativo (App Image)

jpackage \

  --type app-image \

  --name NexusSolairy \

  --app-version 1.0 \

  --input target \

  --main-jar nexus-solairy-1.0-SNAPSHOT.jar \

  --main-class com.example.nexussolairy.Launcher \

  --dest dist
```

10.5 Ejecución de la aplicación empaquetada

```
# Linux
```

```
./dist/NexusAntiquus/bin/NexusSolairy
```

```
# Windows
```

```
dist\NexusAntiquus\NexusSolairy.exe
```

```
# macOS
```

```
open dist/NexusSolairy.app
```

11. Glosario de términos

Término	Definición
AST	Árbol de Sintaxis Abstracta; representación jerárquica simplificada de la estructura lógica del programa
SRA	Simulador/trazado del autómata de análisis sintáctico (pila del parser)
Scope / Ámbito	Región del programa en la que un identificador declarado es visible y accesible
Type Checking	Verificación de la compatibilidad de tipos entre operandos de una operación o asignación
Token	Unidad léxica mínima reconocida por el lexer (palabra reservada, símbolo, literal, identificador)
Shift / Reduce / Accept	Acciones fundamentales del algoritmo de análisis sintáctico ascendente/descendente sobre la pila

Término	Definición
Transpilador	Programa que traduce código fuente de un lenguaje a otro de un nivel de abstracción similar

Anexo A. Gramática completa (ANTLR4)

A continuación se referencia el conjunto de gramáticas completas para los tres lenguajes objetivo del sistema (ZetarianoLexer/ZetarianoParser, YLexer/YParser y PigLatinLexer/PigLatinParser) ubicadas en src/main/antlr4/com/example/nexusantiquus/.

A.1 Gramáticas de Zetariano (ZetarianoLexer.g4 y ZetarianoParser.g4)

```
// Gramática Léxica y Sintáctica de Zetariano

// comentarios y espacios

LINE_COMMENT : '/' ~[\r\n]* -> channel(HIDDEN) ;

BLOCK_COMMENT : '##' .*? '##' -> channel(HIDDEN) ;

WS          : [ \t\r\n]+ -> channel(HIDDEN) ;

// encabezados

VARIABLES : 'VARIABLES' ;

MUNERA    : 'MUNERA'    ;

MAIOR     : 'MAIOR'     ;

FINIS_PROG : 'FINIS'     ;

// declaraciones

ESTO       : 'esto'      ;

SERIES     : 'series'    ;

STRUCTURA : 'estructura' ;

FINIS      : 'finis'     ;

// control de flujo

SI         : 'si'        ;

ALITER     : 'aliter'    ;
```

```
DUM      : 'dum' ;

FACERE   : 'facere' ;

PER       : 'per' ;

PERGE     : 'perge' ;

INTERRUMPE : 'interrumpe' ;

// funciones

ACTIO    : 'actio' ;

RATIO    : 'ratio' ;

REDDERE  : 'reddere' ;

// primitivos

NUMERUS  : 'numerus' ;

TEXTUM   : 'textum' ;

DECIMALIS : 'decimalis' ;

LITTERA  : 'littera' ;

BOOL     : 'bool';

// booleanos

VERUM    : 'verum' ;

FALSUS   : 'falsus' ;

// negacion universal

NON      : 'non' ;

// operadores logicos

EQ       : '==' ;

NEQ      : '!=' ;
```

```
LE    : '<=' ;

GE    : '>=' ;

AND   : '&&' ;

OR    : '||' ;

// manejo de valor

INC   : '++' ;

DEC   : '--' ;

// entrada

READ  : '<<' ;

// SALIDA

PRINT : '>>' ;

// operadores logicos de valor bajo

LT    : '<' ;

GT    : '>' ;

// operadores aritmeticos

PLUS  : '+' ;

MINUS : '-' ;

MULT  : '*' ;

DIV   : '/' ;

// asignacion

ASSIGN : '=' ;

// puntuacion

COLON : ':' ;
```

```

SEMI : ',' ;

COMMA : ',' ;

DOT : '.' ;

// contenedores

LBRACE : '{' ;

RBRACE : '}' ;

LBRACK : '[' ;

RBRACK : ']' ;

LPAREN : '(' ;

RPAREN : ')' ;

// datos primitivos del sistema

DECIMAL : [0-9]+ '.' [0-9]+ ;

NUMBER : [0-9]+ ;

STRING : '"' (ESC | ~["\\r\n])* '"' ;

CHAR : '\' (ESC | ~["\\r\n]) '\'' ;

// fragmentacion en cadenas

fragment ESC : '\\' [btnrf"\\] ;

// identificador

IDENTIFIER : [a-zA-Z_][a-zA-Z0-9_]* ;

```

A.2 Gramáticas de Y? (YLexer.g4 y YParser.g4)

```
// Gramática Léxica y Sintáctica del lenguaje Y?
```

```

options {

    tokenVocab = LatinusLexer;

}

// inicio del programa

init : program EOF;

// =====

// FUNCTION AND PROGRAM

// =====

program : variableSection? functionSection? mainSection FINIS_PROG SEMI

        ;

// =====

// MAIN

// =====

mainSection : MAIOR GT statement*

        ;

// =====

// DECLARACIONES

// =====

variableSection : VARIABLES GT declaration*

```

;

declaration : variableDeclaration

| arrayDeclaration

| structDeclaration

;

localDeclaration : variableDeclaration

| arrayDeclaration

| structDeclaration

;

variableDeclaration : ESTO IDENTIFIER COLON type expression? SEMI

| ESTO IDENTIFIER COLON booleanLiteral SEMI

| ESTO IDENTIFIER COLON IDENTIFIER structInit

| ESTO IDENTIFIER COLON IDENTIFIER expression SEMI

;

arrayDeclaration : SERIES IDENTIFIER LBRAK expression RBRAK COLON type
arrayInit? SEMI

| SERIES IDENTIFIER LBRAK expression RBRAK COLON arrayInit SEMI

;

arrayInit : LBRAK expressionList RBRAK

;

expressionList : expression (COMMA expression)*

;

structDeclaration : STRUCTURA IDENTIFIER LBRACE structField* RBRACE FINIS
SEMI

;

structField : ESTO IDENTIFIER COLON type (COMMA | SEMI)?
| SERIES IDENTIFIER COLON type (COMMA | SEMI)?

;

structInit : LBRACE structFieldInit (COMMA structFieldInit)* RBRACE

;

structFieldInit : IDENTIFIER COLON IDENTIFIER LBRACK expression RBRACK
| IDENTIFIER COLON expression

;

type : NUMERUS

| TEXTUM

| DECIMALIS

| LITTERA

| BOOL

| IDENTIFIER

;

// =====

// FUNCTION

// =====

functionSection : MUNERA GT functionDeclaration*

;

functionDeclaration : voidFunction

| returnFunction

;

voidFunction : ACTIO IDENTIFIER LPAREN parameterList? RPAREN functionBody
FINIS SEMI

;

returnFunction : RATIO type IDENTIFIER LPAREN parameterList? RPAREN
functionBody FINIS SEMI

;

parameterList : parameter (COMMA parameter)*

;

parameter : ESTO IDENTIFIER COLON type

;

functionBody : LBACE functionVarSection? statement* RBACE

;

functionVarSection : VARIABLES LBACE declaration* RBACE

;

// =====

// STATEMENTS AND CONTROL STRUCTURES

// =====

statement : localDeclaration

| assignmentStatement

| ioStatement

| ifStatement

| whileStatement

| doWhileStatement

| forStatement

| jumpStatement

| returnStatement

| block

| expressionStatement

;

assignmentStatement : leftValue ASSIGN expression SEMI

;

leftValue : IDENTIFIER

| IDENTIFIER LBRACK expression RBRACK

| IDENTIFIER DOT IDENTIFIER

| IDENTIFIER DOT IDENTIFIER LBRACK expression RBRACK

;

ioStatement : printStatement

| readStatement

;

printStatement : PRINT expression (PRINT expression)* SEMI

;

readStatement : READ

| IDENTIFIER READ

;

ifStatement : SI LPAREN expression RPAREN block

(ALITER LPAREN expression RPAREN block)*

(ALITER block)?

FINIS SEMI

;

whileStatement : DUM LPAREN expression RPAREN block FINIS SEMI

;

doWhileStatement : FACERE block DUM LPAREN expression RPAREN SEMI

;

forStatement : PER LPAREN forInit SEMI expression SEMI expression RPAREN block

;

forInit : ESTO IDENTIFIER COLON type expression?

| ESTO IDENTIFIER COLON booleanLiteral

| IDENTIFIER ASSIGN expression

;

jumpStatement : INTERRUPE SEMI

| PERGE SEMI

;

returnStatement : REDDERE expression? SEMI

;

block : LBRACE statement* RBRACE

;

expressionStatement : expression SEMI

;

// =====

// EXPRESIONES

// =====

expression : orExpression

;

orExpression : andExpression (OR andExpression)*

;

andExpression : relationalExpression (AND relationalExpression)*

;

relationalExpression : additiveExpression ((EQ | NEQ | LE | GE | LT | GT)
additiveExpression)*

;

additiveExpression : multiplicativeExpression ((PLUS | MINUS) multiplicativeExpression)*

;

multiplicativeExpression : unaryExpression ((MULT | DIV) unaryExpression)*

;

unaryExpression : NON unaryExpression

| MINUS unaryExpression

| postfixExpression

;

postfixExpression : primaryExpression (INC | DEC)?

;

primaryExpression : literal

| IDENTIFIER

| IDENTIFIER LBRACK expression RBRACK

| IDENTIFIER DOT IDENTIFIER

| IDENTIFIER DOT IDENTIFIER LBRACK expression RBRACK

| IDENTIFIER LPAREN argumentList? RPAREN

| LPAREN expression RPAREN

| structLiteral

;

structLiteral : LBACE structFieldInit (COMMA structFieldInit)* RBACE

;

argumentList : expression (COMMA expression)*

;

literal : NUMBER

| DECIMAL

| STRING

| CHAR

| VERUM

| FALSUS

;

booleanLiteral : VERUM

| FALSUS

| expression

;

A.3 Gramáticas de PigLatin (PigLatinLexer.g4 y PigLatinParser.g4)

```
// Gramática Léxica y Sintáctica de PigLatin
```

Anexo B. Diagrama Extra

B.1 Diagrama de estados — Pila del parser (SRA)

